

Week 1

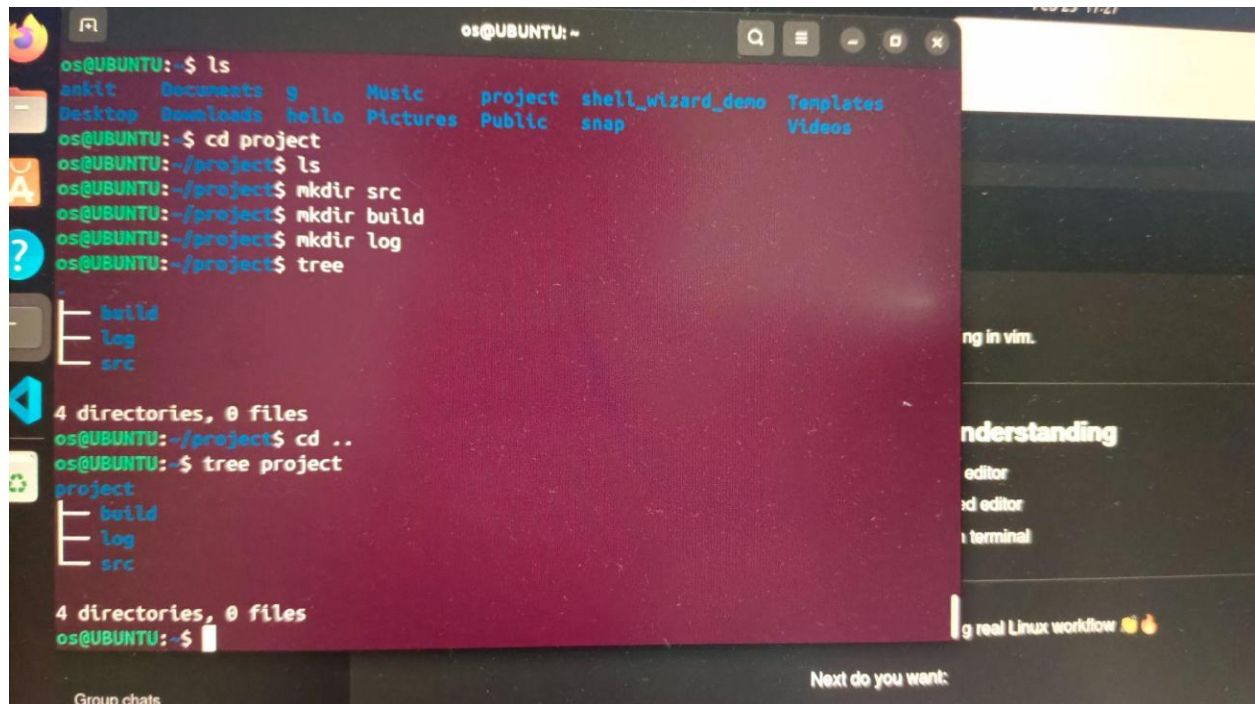
Assignment 1

Topic

1. Create a directory structure like

```
|--project/  
    |--src/  
    |--build/  
    |--log/
```

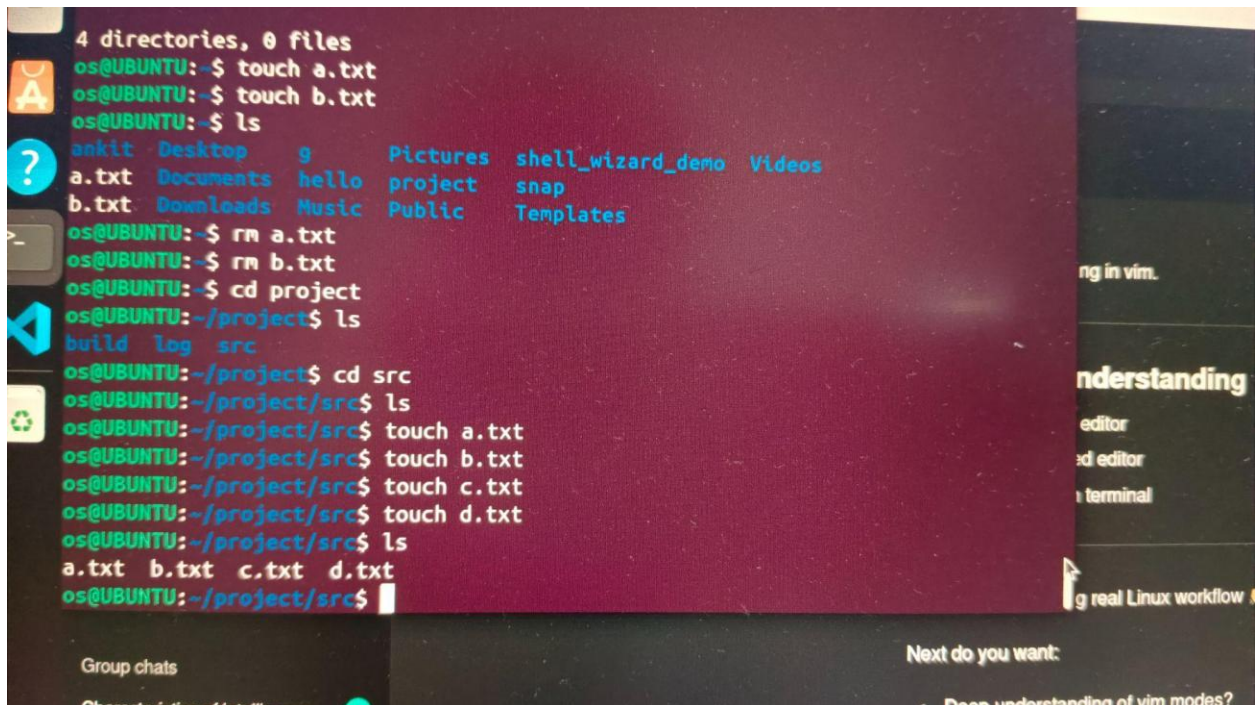
Ans



A terminal window on an Ubuntu system showing the creation of a directory structure. The user starts in the root directory and lists files, including a 'project' directory. They then move into 'project' and create three subdirectories: 'src', 'build', and 'log'. Finally, they use the 'tree' command to verify the structure, showing 'project' containing 'build', 'log', and 'src'.

```
os@UBUNTU:~$ ls  
ankit  Documents  g          Music    project  shell_wizard_demo  Templates  
Desktop Downloads hello      Pictures  Public    snap              Videos  
os@UBUNTU:~$ cd project  
os@UBUNTU:~/project$ ls  
os@UBUNTU:~/project$ mkdir src  
os@UBUNTU:~/project$ mkdir build  
os@UBUNTU:~/project$ mkdir log  
os@UBUNTU:~/project$ tree  
.  
├── build  
├── log  
└── src  
  
4 directories, 0 files  
os@UBUNTU:~/project$ cd ..  
os@UBUNTU:~$ tree project  
project  
├── build  
├── log  
└── src  
  
4 directories, 0 files  
os@UBUNTU:~$
```

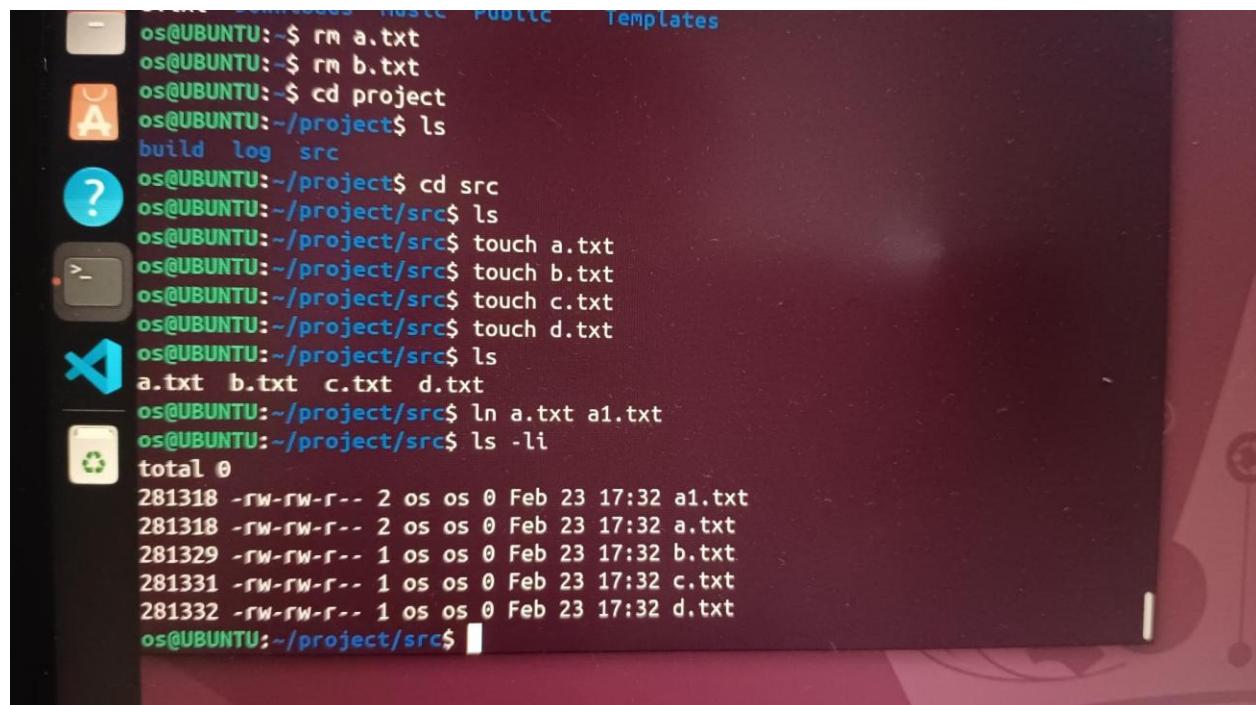
2. Create files inside `src/`

A terminal window on a Linux system. The user is in the root directory and creates two files, a.txt and b.txt. Then they list the directory contents, showing a.txt, b.txt, and several directories. They then remove a.txt and b.txt, and change the directory to /project. They list the contents of /project, showing build, log, and src directories. They then change the directory to /project/src and create four files: a.txt, b.txt, c.txt, and d.txt. Finally, they list the contents of /project/src, showing all four files.

```
4 directories, 0 files
os@UBUNTU:~$ touch a.txt
os@UBUNTU:~$ touch b.txt
os@UBUNTU:~$ ls
ankit Desktop g Pictures shell_wizard_demo Videos
a.txt Documents hello project snap
b.txt Downloads Music Public Templates
os@UBUNTU:~$ rm a.txt
os@UBUNTU:~$ rm b.txt
os@UBUNTU:~$ cd project
os@UBUNTU:~/project$ ls
build log src
os@UBUNTU:~/project$ cd src
os@UBUNTU:~/project/src$ ls
os@UBUNTU:~/project/src$ touch a.txt
os@UBUNTU:~/project/src$ touch b.txt
os@UBUNTU:~/project/src$ touch c.txt
os@UBUNTU:~/project/src$ touch d.txt
os@UBUNTU:~/project/src$ ls
a.txt b.txt c.txt d.txt
os@UBUNTU:~/project/src$
```

3. Create:

a. One hard link

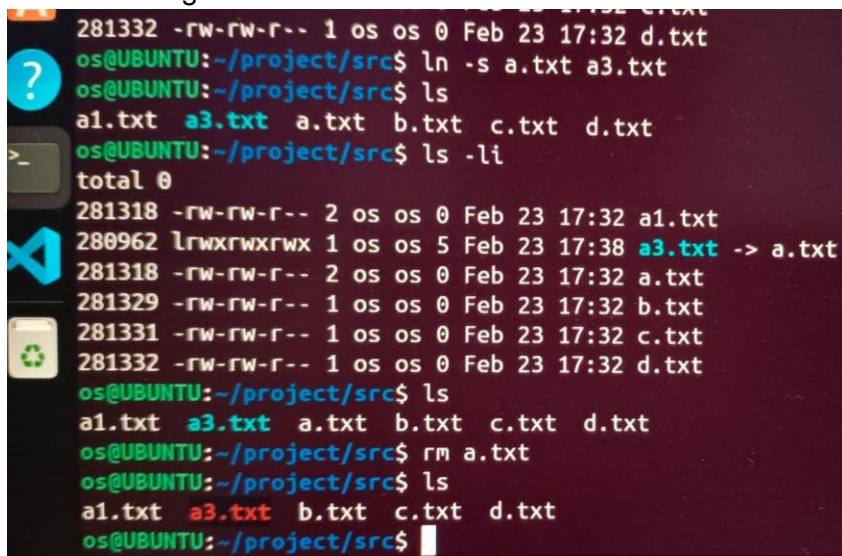
A terminal window on a Linux system. The user is in the root directory and removes a.txt and b.txt. They then change the directory to /project and list the contents, showing build, log, and src directories. They then change the directory to /project/src and create four files: a.txt, b.txt, c.txt, and d.txt. They then list the contents of /project/src, showing all four files. Finally, they create a hard link a1.txt to a.txt and list the contents of /project/src with the -li option, showing the link counts for each file.

```
os@UBUNTU:~$ rm a.txt
os@UBUNTU:~$ rm b.txt
os@UBUNTU:~$ cd project
os@UBUNTU:~/project$ ls
build log src
os@UBUNTU:~/project$ cd src
os@UBUNTU:~/project/src$ ls
os@UBUNTU:~/project/src$ touch a.txt
os@UBUNTU:~/project/src$ touch b.txt
os@UBUNTU:~/project/src$ touch c.txt
os@UBUNTU:~/project/src$ touch d.txt
os@UBUNTU:~/project/src$ ls
a.txt b.txt c.txt d.txt
os@UBUNTU:~/project/src$ ln a.txt a1.txt
os@UBUNTU:~/project/src$ ls -li
total 0
281318 -rw-rw-r-- 2 os os 0 Feb 23 17:32 a1.txt
281318 -rw-rw-r-- 2 os os 0 Feb 23 17:32 a.txt
281329 -rw-rw-r-- 1 os os 0 Feb 23 17:32 b.txt
281331 -rw-rw-r-- 1 os os 0 Feb 23 17:32 c.txt
281332 -rw-rw-r-- 1 os os 0 Feb 23 17:32 d.txt
os@UBUNTU:~/project/src$
```

b. One symbolic link

```
os@UBUNTU:~/project/src$ touch c.txt
os@UBUNTU:~/project/src$ touch d.txt
os@UBUNTU:~/project/src$ ls
a.txt b.txt c.txt d.txt
os@UBUNTU:~/project/src$ ln a.txt a1.txt
os@UBUNTU:~/project/src$ ls -li
total 0
281318 -rw-rw-r-- 2 os os 0 Feb 23 17:32 a1.txt
281318 -rw-rw-r-- 2 os os 0 Feb 23 17:32 a.txt
281329 -rw-rw-r-- 1 os os 0 Feb 23 17:32 b.txt
281331 -rw-rw-r-- 1 os os 0 Feb 23 17:32 c.txt
281332 -rw-rw-r-- 1 os os 0 Feb 23 17:32 d.txt
os@UBUNTU:~/project/src$ ln -s a.txt a3.txt
os@UBUNTU:~/project/src$ ls
a1.txt a3.txt a.txt b.txt c.txt d.txt
os@UBUNTU:~/project/src$ ls -li
total 0
281318 -rw-rw-r-- 2 os os 0 Feb 23 17:32 a1.txt
280962 lrwxrwxrwx 1 os os 5 Feb 23 17:38 a3.txt -> a.txt
281318 -rw-rw-r-- 2 os os 0 Feb 23 17:32 a.txt
281329 -rw-rw-r-- 1 os os 0 Feb 23 17:32 b.txt
281331 -rw-rw-r-- 1 os os 0 Feb 23 17:32 c.txt
281332 -rw-rw-r-- 1 os os 0 Feb 23 17:32 d.txt
os@UBUNTU:~/project/src$
```


4. Delete the original file.



A terminal window on a Linux system (Ubuntu) showing a series of commands and their outputs in the directory `~/project/src`. The commands and outputs are as follows:

```
os@UBUNTU:~/project/src$ ln -s a.txt a3.txt
os@UBUNTU:~/project/src$ ls
a1.txt  a3.txt  a.txt  b.txt  c.txt  d.txt
os@UBUNTU:~/project/src$ ls -li
total 0
281318 -rw-rw-r-- 2 os os 0 Feb 23 17:32 a1.txt
280962 lrwxrwxrwx 1 os os 5 Feb 23 17:38 a3.txt -> a.txt
281318 -rw-rw-r-- 2 os os 0 Feb 23 17:32 a.txt
281329 -rw-rw-r-- 1 os os 0 Feb 23 17:32 b.txt
281331 -rw-rw-r-- 1 os os 0 Feb 23 17:32 c.txt
281332 -rw-rw-r-- 1 os os 0 Feb 23 17:32 d.txt
os@UBUNTU:~/project/src$ ls
a1.txt  a3.txt  a.txt  b.txt  c.txt  d.txt
os@UBUNTU:~/project/src$ rm a.txt
os@UBUNTU:~/project/src$ ls
a1.txt  a3.txt  b.txt  c.txt  d.txt
os@UBUNTU:~/project/src$
```

1. Observe behavior using:

- o `ls -li`

```

281318 -rw-rw-r-- 2 os os 0 Feb 23 17:32 a1.txt
280962 lrwxrwxrwx 1 os os 5 Feb 23 17:38 a3.txt -> a.txt
281318 -rw-rw-r-- 2 os os 0 Feb 23 17:32 a.txt
281329 -rw-rw-r-- 1 os os 0 Feb 23 17:32 b.txt
281331 -rw-rw-r-- 1 os os 0 Feb 23 17:32 c.txt
281332 -rw-rw-r-- 1 os os 0 Feb 23 17:32 d.txt
os@UBUNTU:~/project/src$ ls
a1.txt a3.txt a.txt b.txt c.txt d.txt
os@UBUNTU:~/project/src$ rm a.txt
os@UBUNTU:~/project/src$ ls
a1.txt a3.txt b.txt c.txt d.txt
os@UBUNTU:~/project/src$ ls -li
total 0
281318 -rw-rw-r-- 1 os os 0 Feb 23 17:32 a1.txt
280962 lrwxrwxrwx 1 os os 5 Feb 23 17:38 a3.txt -> a.txt
281329 -rw-rw-r-- 1 os os 0 Feb 23 17:32 b.txt
281331 -rw-rw-r-- 1 os os 0 Feb 23 17:32 c.txt
281332 -rw-rw-r-- 1 os os 0 Feb 23 17:32 d.txt
os@UBUNTU:~/project/src$

```

o stat

```

os@UBUNTU:~/project$ cd src
os@UBUNTU:~/project/src$ ls
a1.txt a3.txt b.txt c.txt d.txt
os@UBUNTU:~/project/src$ stat a3.txt
  File: a3.txt -> a.txt
  Size: 5          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,2      Inode: 280962       Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/      os)   Gid: ( 1000/      os)
Access: 2026-02-23 17:38:16.284464448 +0000
Modify: 2026-02-23 17:38:14.673370971 +0000
Change: 2026-02-23 17:38:14.673370971 +0000
 Birth: 2026-02-23 17:38:14.673370971 +0000
os@UBUNTU:~/project/src$ stat a1.txt
  File: a1.txt
  Size: 0          Blocks: 0          IO Block: 4096   regular empty file
Device: 8,2      Inode: 281318       Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/      os)   Gid: ( 1000/      os)
Access: 2026-02-23 17:32:40.754276579 +0000
Modify: 2026-02-23 17:32:40.754276579 +0000
Change: 2026-02-23 17:40:11.566538768 +0000
 Birth: 2026-02-23 17:32:40.754276579 +0000
os@UBUNTU:~/project/src$

```

ng in vim.

nderstanding

editor
d editor
terminal

g real Linux workflow 🍌

Group chats

Next do you want:

Q. Explain inode behavior

Ans. inode is assigned to each file and it is unique for each file.

But in the hard link we can see that 2 files have the same inode number. This was because the inode number is always one but it's name can be more than one or two and that name is pointed to the same inode.

Q. Explain why one link survives and one breaks

Ans. One link Survives and one breaks because:-

1. Hard link survive because when we create a copy from the parent then copy was not pointed to parent it was pointed to inode that's why when parent was deleted it was not broken
2. In Symbolic link the new file that was broke when it's parent was deleted as in symbolic link the copy file is not pointed to inode it was pointed to it's parent that's why it broke.

Assignment 2

You are given:

Permission denied

Simulate 3 different causes:

- Missing execute on directory

```
os@UBUNTU:~/ankit$ cd
os@UBUNTU:~$ mkdir OS
os@UBUNTU:~$ cd OS
os@UBUNTU:~/OS$ mkdir p
os@UBUNTU:~/OS$ ls
p
os@UBUNTU:~/OS$ chmod -R u-x p
os@UBUNTU:~/OS$ ls
p
os@UBUNTU:~/OS$ cd p
bash: cd: p: Permission denied
os@UBUNTU:~/OS$
```

In this I remove the permission for execute on P folder

```

p
os@UBUNTU:~/OS$ chmod -R u-x p
os@UBUNTU:~/OS$ ls
p
os@UBUNTU:~/OS$ cd p
bash: cd: p: Permission denied
os@UBUNTU:~/OS$ chmod -R u+x p
os@UBUNTU:~/OS$ cd p
os@UBUNTU:~/OS/p$ ls p
ls: cannot access 'p': No such file or directory
os@UBUNTU:~/OS/p$ ls
os@UBUNTU:~/OS/p$

```

Regive the permission to execute p

- Wrong ownership

```

os@UBUNTU:~/OS/p$ ls
owner
os@UBUNTU:~/OS/p$ nano owner
os@UBUNTU:~/OS/p$ ls
owner
os@UBUNTU:~/OS/p$ chmod 000 owner
chmod: changing permissions of 'owner': Operation not permitted
os@UBUNTU:~/OS/p$ sudo chown os owner
os@UBUNTU:~/OS/p$ ls
owner
os@UBUNTU:~/OS/p$ chmod 000 owner
os@UBUNTU:~/OS/p$

```

In this I change the owner to root and then re back ownership to os user

- Incorrect file mode


```

os@UBUNTU:~/OS/p$ sudo chown os owner
os@UBUNTU:~/OS/p$ ls
owner
os@UBUNTU:~/OS/p$ chmod 000 owner
os@UBUNTU:~/OS/p$ cat owner
cat: owner: Permission denied
os@UBUNTU:~/OS/p$ nano owner
os@UBUNTU:~/OS/p$ chmod 777 owner
os@UBUNTU:~/OS/p$ cat owner
XHello everyone
os@UBUNTU:~/OS/p$

```

Change the permission of owner and then reback it

Then fix them without using `sudo` unless necessary.

Sudo was only used in changing the ownership.

🎯 Assignment 3: Safe Backup Workflow

Create a script-less manual workflow that:

- Archives a directory

```

os@UBUNTU:~$ rm Compress_OS
os@UBUNTU:~$ l
ankit/      Documents/  hello/     Pictures/  shell_wizard_demo/
ansh.compress* Downloads/  Music/     project/   snap/
Desktop/    g/         OS/        Public/    Templates/
os@UBUNTU:~$ ls
ankit      Documents  hello     Pictures  shell_wizard_demo
ansh.compress Downloads  Music     project   snap
Desktop    g         OS        Public    Templates
os@UBUNTU:~$ tar cvf Archive_OS OS
OS/
OS/p/
OS/p/owner
OS/p/gh/
OS/p/hf/
os@UBUNTU:~$ ls
ankit      Desktop    g         OS        Public    Templates
ansh.compress Documents  hello     Pictures  shell_wizard_demo
Archive_OS Downloads  Music     project   snap
os@UBUNTU:~$

```

IN this I archive the OS folder by the name Archive_OS

- Compresses it

```
os@UBUNTU:~$ stat OS
  File: OS
  Size: 4096          Blocks: 8          IO Block: 4096   directory
Device: 8,2      Inode: 524409      Links: 3
Access: (0775/drwxrwxr-x)  Uid: ( 1000/      os)   Gid: ( 1000/      os)
Access: 2026-03-30 13:35:24.484160060 +0000
Modify: 2026-03-30 13:35:22.308135930 +0000
Change: 2026-03-30 13:35:22.308135930 +0000
 Birth: 2026-03-30 13:34:41.448700203 +0000
os@UBUNTU:~$
```

```
os@UBUNTU:~$ ls
ankit      Desktop   g         OS         Public      Templates
ansh.compress Documents hello Pictures shell_wizard_demo
Archive_OS Downloads Music  project    snap
os@UBUNTU:~$ zip compress_OS OS
  adding: OS/ (stored 0%)
os@UBUNTU:~$ stat compress_OS
stat: cannot statx 'compress_OS': No such file or directory
os@UBUNTU:~$ stat /compress_OS
stat: cannot statx '/compress_OS': No such file or directory
os@UBUNTU:~$ sudo stat compress_OS
[sudo] password for os:
stat: cannot statx 'compress_OS': No such file or directory
os@UBUNTU:~$ ls
ankit      Desktop   hello     project     Templates
ansh.compress Documents Music   Public
Archive_OS Downloads OS        shell_wizard_demo
compress_OS.zip g         Pictures  snap
os@UBUNTU:~$ gzip -k OS
gzip: OS is a directory -- ignored
os@UBUNTU:~$ ls
```

```

os@UBUNTU:~$ gzip -k OS
gzip: OS is a directory -- ignored
os@UBUNTU:~$ ls
ankit          Desktop      hello       project      Templates
ansh.compress  Documents   Music       Public
Archive_OS     Downloads   OS          shell_wizard_demo
compress_OS.zip g           Pictures    snap
os@UBUNTU:~$ size compress_OS.zip
size: compress_OS.zip: file format not recognized
os@UBUNTU:~$ ls -lh compress_OS.zip
-rw-rw-r-- 1 os os 156 Mar 30 14:21 compress_OS.zip
os@UBUNTU:~$ stat compress_OS.zip
  File: compress_OS.zip
  Size: 156          Blocks: 8          IO Block: 4096   regular file
Device: 8,2      Inode: 294109      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/      os)   Gid: ( 1000/      os)
Access: 2026-03-30 14:24:34.610352445 +0000
Modify: 2026-03-30 14:21:52.296385805 +0000
Change: 2026-03-30 14:21:52.307385705 +0000
 Birth: 2026-03-30 14:21:52.296385805 +0000
os@UBUNTU:~$

```

In this I make compress the OS and compressed file name compress_OS.zip

- Stores it with timestamp
- Verifies size before and after

🔗 Assignment 4: Shell Personalization & Alias Behavior

Task:

1. Create 5 useful aliases:
 - One for `ls`

```

os@UBUNTU:~$ alias l='ls'
os@UBUNTU:~$ l
ankit      Desktop  hello    project  Templates
anish.compress Documents Music    Public
Archive_OS Downloads OS        shell_wizard_demo
compress_OS.zip g        Pictures snap
os@UBUNTU:~$

```

-
- One for navigation

```

os@UBUNTU:~$ alias l='ls'
os@UBUNTU:~$ l
ankit      Desktop  hello    project  Templates
anish.compress Documents Music    Public
Archive_OS Downloads OS        shell_wizard_demo
compress_OS.zip g        Pictures snap
os@UBUNTU:~$ alias c='cd'
os@UBUNTU:~$ c OS
os@UBUNTU:~/OS$ ls
os@UBUNTU:~/OS$ l
os@UBUNTU:~/OS$

```

- One for search


```

os@UBUNTU:~$ alias l='ls'
os@UBUNTU:~$ l
ankit      Desktop  hello    project  Templates
anish.compress Documents Music    Public
Archive_OS Downloads OS       shell_wizard_demo
compress_OS.zip g        Pictures snap
os@UBUNTU:~$ alias c='cd'
os@UBUNTU:~$ c OS
os@UBUNTU:~/OS$ ls
os@UBUNTU:~/OS$ l
os@UBUNTU:~/OS$ alias search='grep -i'
os@UBUNTU:~/OS$ search hello p
grep: p: Is a directory

```

- One for git (if installed)
- One custom productivity shortcut

```

os@UBUNTU:~/OS$ alias search='grep -i'
os@UBUNTU:~/OS$ search hello p
grep: p: Is a directory
os@UBUNTU:~/OS$ alias f='mkdir'
os@UBUNTU:~/OS$ f ankit
os@UBUNTU:~/OS$ l
ankit  p
os@UBUNTU:~/OS$

```

2. Make them:

- Work in current session
- Persist across terminal restarts

```
compress_05.zip g/ Pictures/ snap/
os@UBUNTU:~$ f ankit
f: command not found
os@UBUNTU:~$
```

3. Verify:

- `type <alias_name>`
- `alias`
- Restart terminal and test again
- Create an alias that breaks something intentionally.
Example:

```
alias rm='rm -i'
```

- Then override it temporarily using:

```
\rm
```

```

Archieve_OS      Downloads  OS      shell_wizard_demo
compress_OS.zip  g          Pictures snap
os@UBUNTU:~$ cd ankit
os@UBUNTU:~/ankit$ ls
a  ankit.txt  'ankit.txt\\'  b  h
os@UBUNTU:~/ankit$ rm a
rm: remove regular empty file 'a'? y
os@UBUNTU:~/ankit$ ls
ankit.txt  'ankit.txt\\'  b  h
os@UBUNTU:~/ankit$ alias rm='mkdir'
os@UBUNTU:~/ankit$ rm ankit
os@UBUNTU:~/ankit$ ls
ankit  ankit.txt  'ankit.txt\\'  b  h
os@UBUNTU:~/ankit$ \rm
rm: missing operand
Try 'rm --help' for more information.
os@UBUNTU:~/ankit$ \rm ankit
rm: cannot remove 'ankit': Is a directory
os@UBUNTU:~/ankit$ ls
ankit  ankit.txt  'ankit.txt\\'  b  h
os@UBUNTU:~/ankit$ \rm ankit.txt
os@UBUNTU:~/ankit$ ls
ankit  'ankit.txt\\'  b  h
os@UBUNTU:~/ankit$

```

Deliverable:

- Explain why `\command` bypasses alias

I think when we use `'\'` this then system go to root process.

- Explain why aliases don't work inside non-interactive scripts
- Show difference between alias and function

